

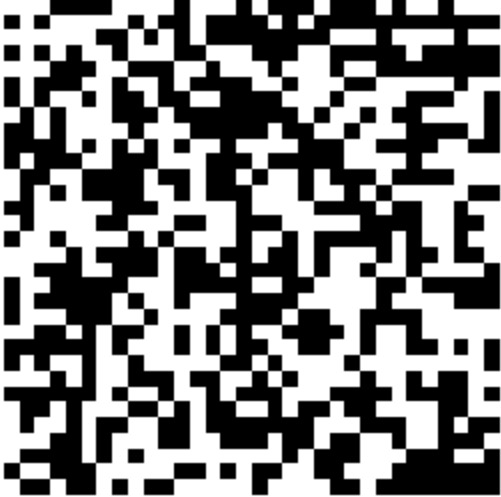
1. Penjelasan Kode

Kode	Penjelasan
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt</pre>	Melakukan import <i>library</i> dasar yang dibutuhkan. cv2 (OpenCV) untuk memproses citra, numpy untuk komputasi operasi matriks/array, dan matplotlib untuk menampilkan hasil visualisasi gambar.
<pre>host = cv2.imread('wajah.jpg', cv2.IMREAD_GRAYSCALE) wm = cv2.imread('watermark.png', cv2.IMREAD_GRAYSCALE)</pre>	Membaca file gambar <i>host</i> (wajah) dan <i>watermark</i> , lalu mengubahnya langsung menjadi format <i>Grayscale</i> (hitam-putih) agar pemrosesan fokus pada intensitas (<i>luminance</i>) tanpa memproses <i>channel</i> warna lain.
<pre>wm = cv2.resize(wm, (host.shape[1], host.shape[0])) wm_float = np.float32(wm) / 255.0</pre>	Menyesuaikan dimensi gambar <i>watermark</i> agar sama persis dengan dimensi gambar <i>host</i> . Nilai piksel <i>watermark</i> dinormalisasi menjadi rentang 0 hingga 1.
<pre>dct_host = apply_dct(host_float) dct_wm = apply_dct(wm_float)</pre>	Menerapkan <i>Discrete Cosine Transform</i> (DCT) untuk mengubah citra dari domain spasial (piksel) ke domain frekuensi. Penyisipan di domain frekuensi membuat <i>watermark</i> lebih tangguh (<i>robust</i>) terhadap kompresi.
<pre>dct_watermarked = dct_host + (alpha * wm_float)</pre>	Menyisipkan <i>watermark</i> secara aditif ke dalam koefisien DCT gambar <i>host</i> menggunakan faktor kekuatan/skala (<i>alpha</i>). Semakin besar <i>alpha</i> , semakin kuat <i>watermark</i> bertahan, tapi gambar asli akan semakin terdistorsi.

<pre>watermarked_float = apply_idct(dct_watermarked) watermarked_img = np.uint8(...)</pre>	Menerapkan <i>Inverse Discrete Cosine Transform</i> (IDCT) untuk mengembalikan matriks frekuensi yang sudah disisipi <i>watermark</i> kembali menjadi bentuk citra spasial (piksel 0-255).
<pre>encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), qf] _, encimg = cv2.imencode('.jpg', watermarked_img, encode_param) decimg = cv2.imdecode(encimg, cv2.IMREAD_GRAYSCALE)</pre>	Melakukan simulasi kompresi citra ke format JPEG. Gambar di-<i>encode</i> ke dalam memori dengan parameter <i>Quality Factor</i> (qf) tertentu, lalu di-<i>decode</i> kembali untuk melihat efek hilangnya data (<i>lossy compression</i>).
<pre>extracted = (dct_wm - dct_host) / alpha</pre>	Mengekstrak kembali <i>watermark</i> dari gambar yang telah dikompresi dengan cara mencari selisih koefisien matriks frekuensinya terhadap matriks gambar asli, lalu dibagi dengan faktor alpha.
<pre>qf_list = [100, 90, 70, 50, 20] for qf in qf_list:</pre>	Mendefinisikan daftar nilai <i>Quality Factor</i> (QF) yang akan diuji dan melakukan iterasi (pengulangan) untuk menguji ketahanan <i>watermark</i> pada masing-masing tingkat kompresi tersebut.

2. Hasil Visualisasi

				
QF 20	QF 50	QF 70	QF 90	QF 100

Foto Wajah	Watermark
	Watermark biner 32×32 

3. Analisis Hasil Kinerja Watermark terhadap Kompresi JPEG

Berdasarkan pengujian kompresi JPEG dengan beberapa nilai *Quality Factor* (QF), didapatkan hasil analisis ekstraksi sebagai berikut:

QF	Analisis Kondisi Visual Watermark	Status
100	<i>Watermark</i> diekstrak dengan sangat sempurna tanpa adanya pengurangan kualitas. Pola piksel hitam-putih terlihat utuh layaknya gambar <i>watermark</i> asli. Hal ini terjadi karena QF 100 melakukan kompresi dengan tingkat <i>loss</i> yang sangat minimal, mempertahankan hampir seluruh koefisien	Dapat diekstraksi

	frekuensi tinggi.	
90	Terdapat sedikit degradasi (penurunan kualitas) dibandingkan QF 100, namun secara kasat mata <i>watermark</i> masih sangat bersih, utuh, dan mudah dibaca polanya. Algoritma JPEG mulai membuang sebagian kecil informasi frekuensi tinggi, tetapi tidak merusak letak sisipan <i>watermark</i>.	Dapat diekstraksi
70	Mulai terlihat adanya <i>noise</i> atau bintik-bintik keabu-abuan pada hasil ekstraksi <i>watermark</i>. Kuantisasi matriks JPEG pada QF ini mulai menggeser koefisien frekuensi menengah. Meski demikian, bentuk dan pola <i>watermark</i> secara keseluruhan masih cukup jelas dan dapat dikenali.	Dapat diekstraksi
50	<i>Noise</i> yang muncul semakin banyak, tebal, dan merata. Tepi-tepi pola <i>watermark</i> mulai kabur dan menyatu dengan sekitarnya (<i>blurring</i>). Pada tahap ini, kompresi JPEG membuang lebih banyak data frekuensi, menyebabkan <i>watermark</i> terdegradasi secara signifikan meskipun siluet kasarnya masih sedikit terlihat.	Rusak sebagian

20	<i>Watermark</i> hancur secara total dan berubah menjadi pola blok-blok piksel acak (<i>noise</i> pekat). Nilai QF 20 menerapkan kompresi tingkat tinggi yang memotong paksa (<i>truncate</i>) hampir seluruh koefisien frekuensi menengah dan tinggi menjadi nol (0) melalui tabel kuantisasinya. Akibatnya, informasi <i>watermark</i> yang disisipkan lenyap.	Gagal diekstraksi
----	---	-------------------

Kesimpulan Evaluasi:

Berdasarkan tabel di atas, skema penyisipan *watermark* (berbasis DCT Aditif) ini mampu bertahan dengan baik pada rentang kompresi JPEG ringan hingga menengah. Ketahanan kritis berada pada **QF 50** di mana gambar mulai rusak sebagian. Ketika dikompresi secara ekstrem hingga **QF 20**, *watermark* dipastikan **gagal diekstraksi**.